GAIA Memory v2 Architecture

Status: Active **Version:** 2.0 **Date:** 2026-04-01 **Scope:** Unified "second brain" memory system -- semantic recall, extraction, consolidation, temporal awareness **Files:** 3 new (`memory_store.py`, `memory.py`, `discovery.py`), 1 edit (`agent.py`) **Dependencies:** stdlib (`sqlite3`, `threading`, `json`, `re`, `uuid`), required (`faiss-cpu`, `numpy`)

Design Philosophy

The agent is a **trusted second brain** -- it remembers everything, surfaces the right thing at the right time, and holds the user accountable to their own commitments. Memory should feel invisible in storage but proactive in recall.

Four principles:

1. **Store automatically** -- conversations, tool calls, errors, preferences, meeting notes, journal entries
2. **Recall naturally** -- hybrid semantic+keyword search, not just exact phrase matching. The LLM decides when to search memory, using its own tools (no forced pre-query step).
3. **Learn continuously** -- LLM extraction from every conversation, error auto-learning, confidence evolution over time
4. **Be temporally aware** -- know what time it is, what's due, surface reminders proactively

Design References

Project	What we took / compared
gaia6 (internal)	FTS5 with AND->OR fallback, Szymkiewicz-Simpson deduplication, BM25 ranking, confidence decay. Simplified: 2 databases -> 1, 8 tools -> 5, 7 categories -> 6.
General agent memory literature	Confidence scoring (+0.02 on recall, x0.9 decay), temporal awareness (`due_at`/`reminded_at`), sensitivity classification.
CoALA framework	Four-tier cognitive architecture alignment (working, episodic, semantic, procedural). See Memory Tiers section.

Key design decision: The frozen prefix approach keeps stable content (facts, preferences) in a cached system prompt so the LLM inference engine can reuse its KV-cache, while time-sensitive content (current time, upcoming items) is injected per-turn. GAIA implements this with `get_memory_system_prompt()` (stable) + `get_memory_dynamic_context()` (per-turn, prepended to user message).

Memory Tiers (CoALA Alignment)

GAIA's memory maps to the four-tier cognitive architecture from the CoALA framework:

Tier	What it is	GAIA Implementation
Working	Active context window	System prompt (stable prefix) + dynamic context (per-turn suffix)
Episodic	Past events and interactions	`conversations` table -- all turns, FTS5 searchable, consolidation-eligible
Semantic	Facts, knowledge, concepts	`knowledge` table -- hybrid search (vector + BM25), confidence-weighted
Procedural	Skills and learned behaviors	`knowledge(category='error')` + `knowledge(category='skill')` -- injected into stable prefix

The working memory tier is bounded by the LLM's context window. The stable prefix is frozen for KV-cache reuse; the dynamic suffix changes every turn. Episodic memory is the raw conversation log -- immutable, append-only, pruned at 90 days but consolidated to semantic memory before deletion. Semantic memory is the distilled knowledge base -- facts, preferences, skills, errors -- ranked by confidence and surfaced via hybrid search. Procedural memory is a subset of semantic memory that the agent injects into every system prompt: error patterns to avoid and learned workflows to follow.

Architecture Overview

```
+-----------------------------------------------------+
|                     Agent                           |
|                                                     |
|  +--------------+    +---------------------------+  |
|  | MemoryMixin  |    |   Agent.process_query()   |  |
|  |              |    |                           |  |
|  | Hooks into:  |--->| 1. _compose_system_prompt  |  |
|  |  * prompt    |    |    -> inject preferences   |  |
|  |  * tool exec |    |    -> inject error patterns|  |
|  |  * post-query|    |    -> inject skills        |  |
|  |              |    |                           |  |
|  | Exposes:     |    | 2. _execute_tool           |  |
|  |  * remember  |    |    -> auto-log call+result |  |
|  |  * recall    |    |    -> auto-log errors      |  |
|  |  * forget    |    |                           |  |
|  |  * update    |    | 3. after process_query     |  |
|  |  * search    |    |    -> store conversation   |  |
|  |              |    |    -> LLM extraction       |  |
|  +------+-------+    |    -> embed new knowledge  |  |
|         |            +---------------------------+  |
|  +------v-------+                                    |
|  | MemoryStore  |  <- Pure data layer (no Agent deps)|
|  |              |                                    |
|  | Single file: |                                    |
|  | ~/.gaia/     |                                    |
|  |  memory.db   |                                    |
|  +--------------+                                    |
+-----------------------------------------------------+
```

Schema

Single Database: `~/.gaia/memory.db`

One file, three tables. WAL mode for concurrent reads. Schema version 2.

Timestamps

All timestamps use ISO 8601 format with timezone: `YYYY-MM-DDTHH:MM:SS+HH:MM` (e.g., `2026-04-01T14:30:00-07:00`).

This format is:

- **Human-readable** -- the LLM can reason about "last Tuesday" or "yesterday at 3pm"
- **Sortable** -- lexicographic ordering works correctly
- **Timezone-aware** -- critical for users who travel or work across timezones
- **SQL-friendly** -- SQLite's comparison operators work natively on this format

Stored via Python: `datetime.now().astimezone().isoformat()` (local time with UTC offset).

Important caveat: SQLite's built-in `datetime()` function does NOT understand timezone offsets. The temporal queries (`get_upcoming`, etc.) must compare against a Python-generated "now" string passed as a parameter, not

`datetime('now')` in SQL. This is handled in the implementation -- all time comparisons use parameterized queries with Python-computed boundaries.

Complete v2 Schema

```
-- Schema version tracking (for migrations)
CREATE TABLE IF NOT EXISTS schema_version (
    version     INTEGER NOT NULL,
    migrated_at TEXT NOT NULL         -- ISO 8601
);
-- Initialize: INSERT INTO schema_version VALUES (2, <now>);
-- Table 1: knowledge
-- Persistent facts, preferences, learnings -- the "second brain"
CREATE TABLE knowledge (
    id          TEXT PRIMARY KEY,     -- UUID
    category    TEXT NOT NULL,        -- 'fact' | 'preference' |
'error' | 'skill' | 'note' | 'reminder'
    content     TEXT NOT NULL,        -- Human-readable description
    domain      TEXT,                 -- Optional sub-type (e.g.,
'journal', 'meeting:standup', 'deployment')
    source      TEXT NOT NULL DEFAULT 'tool',  -- 'tool' |
'llm_extract' | 'error_auto' | 'user' | 'discovery' |
'consolidation'
    confidence  REAL DEFAULT 0.5,    -- 0.0 to 1.0, decays over
time
    metadata    TEXT,                 -- JSON blob for structured
data
    use_count   INTEGER DEFAULT 0,
    -- Context scoping
    context     TEXT DEFAULT 'global', -- Workspace/context scope
(e.g., 'work', 'personal', 'project-x')
    -- Sensitivity
    sensitive   INTEGER DEFAULT 0,   -- 1 = sensitive (excluded
from system prompt, redacted in logs)
    -- Entity linking
    entity      TEXT,                -- Optional entity reference
(e.g., 'person:sarah_chen', 'app:vscode')
    -- Timestamps
    created_at  TEXT NOT NULL,       -- ISO 8601 with timezone
(when first stored)
    updated_at  TEXT NOT NULL,       -- ISO 8601 (set on every
modification)
    last_used   TEXT,                -- ISO 8601 (set on every
recall)
    -- Temporal awareness
    due_at      TEXT,                -- ISO 8601 (when this becomes
actionable/relevant)
    reminded_at TEXT,                -- ISO 8601 (when agent last
surfaced this to user)
    -- Vector embedding
    embedding   BLOB,                -- float32 vector
(nomic-embed-text-v2-moe-GGUF, 768-dim). NULL = not yet embedded.
    -- Fact lineage (Zep-inspired)
    superseded_by TEXT               -- ID of newer knowledge item
```

```
that replaced this one. NULL = current/active.
);
CREATE INDEX idx_knowledge_due ON knowledge(due_at)
    WHERE due_at IS NOT NULL;
CREATE INDEX idx_knowledge_context ON knowledge(context);
CREATE INDEX idx_knowledge_entity ON knowledge(entity)
    WHERE entity IS NOT NULL;
CREATE INDEX idx_knowledge_sensitive ON knowledge(sensitive)
    WHERE sensitive = 1;
CREATE INDEX idx_knowledge_no_embedding
    ON knowledge(id) WHERE embedding IS NULL;
CREATE INDEX idx_knowledge_superseded ON knowledge(superseded_by)
    WHERE superseded_by IS NOT NULL;
-- FTS5 for knowledge search (standalone, manually synced)
CREATE VIRTUAL TABLE knowledge_fts USING fts5(content, domain,
category);
-- Table 2: conversations
-- Every conversation turn, persistent across sessions
CREATE TABLE conversations (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id      TEXT NOT NULL,
    role            TEXT NOT NULL,        -- 'user' | 'assistant'
    content         TEXT NOT NULL,
    context         TEXT DEFAULT 'global', -- Active context when
this turn occurred
    timestamp       TEXT NOT NULL,        -- ISO 8601 with timezone
(set by Python, not SQL default)
    consolidated_at TEXT                  -- ISO 8601 when
distilled to knowledge. NULL = not yet consolidated.
);
CREATE INDEX idx_conv_session ON conversations(session_id);
CREATE INDEX idx_conv_ts ON conversations(timestamp DESC);
CREATE INDEX idx_conv_context ON conversations(context);
CREATE INDEX idx_conv_not_consolidated
    ON conversations(session_id)
    WHERE consolidated_at IS NULL;
-- FTS5 for conversation search
CREATE VIRTUAL TABLE conversations_fts USING fts5(
    content,
    content=conversations,
    content_rowid=id
);
-- Sync triggers (INSERT/DELETE)
-- Table 3: tool_history
-- Every tool call the agent makes, auto-logged
CREATE TABLE tool_history (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id  TEXT NOT NULL,
    tool_name   TEXT NOT NULL,
    args        TEXT,                 -- JSON (truncated to 500
chars)
    result_summary TEXT,             -- Truncated result (first 500
chars)
    success     INTEGER NOT NULL,    -- 1 = success, 0 = failure
```

```
    error       TEXT,                -- Error message if failed
(truncated to 500 chars)
    duration_ms INTEGER,             -- Execution time
    timestamp   TEXT NOT NULL        -- ISO 8601 with timezone
);
CREATE INDEX idx_tool_name ON tool_history(tool_name);
CREATE INDEX idx_tool_session ON tool_history(session_id);
CREATE INDEX idx_tool_success ON tool_history(success);
CREATE INDEX idx_tool_ts ON tool_history(timestamp DESC);
```

Schema Migrations

Schema version 2 adds two columns vs. v1. Migrations run automatically in `MemoryStore.__init__()`:

```
-- Migration: schema_version 1 -> 2
ALTER TABLE knowledge ADD COLUMN embedding BLOB;
ALTER TABLE conversations ADD COLUMN consolidated_at TEXT;
CREATE INDEX IF NOT EXISTS idx_knowledge_no_embedding
    ON knowledge(id) WHERE embedding IS NULL;
CREATE INDEX IF NOT EXISTS idx_conv_not_consolidated
    ON conversations(session_id) WHERE consolidated_at IS NULL;
UPDATE schema_version SET version = 2, migrated_at = <now>;
```

Knowledge Categories & Domain Conventions

6 Categories

| Category | What it stores | Example | |---|---|---| | `fact` | Things about the user, project, world | "User's project uses React 19 with app router" | | `preference` | How the user wants the agent to behave | "User prefers concise answers", "Always use dark mode" | | `error` | Tool error patterns to avoid | "pip install torch fails without --index-url on this machine" | | `skill` | Learned workflows and patterns | "To deploy: run tests -> build -> push to staging -> verify -> promote" | | `note` | Observations, journal entries, meeting notes | "Standup 2026-04-01: API migration complete" | | `reminder` | Time-sensitive items with `due_at` | "Q2 report due April 15" |

Recommended Domain Naming

Domains are guidelines, not enforced. They provide sub-typing for richer organization:

| Domain | Category | Use case | |--------|----------|----------| | `journal` | note | Daily log / reflection entries | | `journal:YYYY-MM-DD` | note | Dated journal entries for time-based recall | | `meeting` | note | Generic meeting notes | | `meeting:standup` | note | Daily standup notes | | `meeting:1on1` | note | 1:1 meeting notes | | `research` | fact/note | Saved articles, research notes | | `session:<id[:8]>` | note | Auto-generated consolidation summaries | | `habit:<name>` | reminder | Recurring habits/rituals | | `deployment` | skill/error | DevOps workflows and failures |

Entity Naming Conventions

The `entity` field uses a lightweight `type:name` convention for linking knowledge to people, apps, projects, and services:

| Pattern | Example | Use case | |---------|---------|----------| | `person:<name>` | `person:sarah_chen` | Contacts,

colleagues | | `project:<name>` | `project:gaia` | Projects | | `app:<name>` | `app:vscode` | Applications | | `service:<name>` | `service:github` | External services | | `team:<name>` | `team:platform` | Teams | | `site:<name>` | `site:github.com` | Websites |

How entity linking works:

- `recall(entity="person:sarah_chen")` returns everything about Sarah
- `get_by_entity("person:sarah_chen")` in MemoryStore does a direct indexed lookup
- The LLM links entities naturally: when user says "email Sarah about the roadmap," the LLM calls `recall(entity="person:sarah_chen")` to get her email and preferences
- No separate entity table needed -- it's a denormalized tag on knowledge rows
- Multiple entries can share an entity, building a profile over time

| Entity pattern | Example knowledge | |---|---| | `person:sarah_chen` | "Sarah Chen, VP Engineering, sarah@company.com" | | `person:sarah_chen` | "Sarah prefers morning meetings" | | `person:sarah_chen` | "Follow up with Sarah about Q2 roadmap" | | `app:vscode` | "User prefers dark mode, 4-space tabs" | | `service:gmail` | "User's work email is alex@company.com" | | `project:gaia` | "Project uses Python 3.12, uv for package management" |

Knowledge Lifecycle

Knowledge flows through five stages: store, embed, dedup, decay, prune.

Store

New knowledge enters via one of six sources:

| Source | Confidence | How created | |--------|----------|----------| | `tool` | 0.5 | LLM explicitly called `remember()` | | `llm_extract` | 0.4 | Auto-extracted by LLM from conversation (Mem0-style ADD/UPDATE/DELETE) | | `error_auto` | 0.5 | Auto-stored from tool failure | | `user` | 0.8 | Manual creation via dashboard | | `discovery` | 0.4 | System bootstrap scan | | `consolidation` | 0.5 | Distilled from old conversation sessions |

Source is visible in the dashboard and helps users understand why the agent "knows" something.

Embed

After storage, the item is immediately embedded via Lemonade (`nomic-embed-text-v2-moe-GGUF`, 768-dim) and the embedding BLOB is written back. Embedding is a hard requirement ·`init_memory()` validates Lemonade connectivity at startup and raises `RuntimeError` if the embedding endpoint is unreachable. All knowledge items must have embeddings; items without embeddings (e.g., from a schema migration) are backfilled on startup before the system becomes operational.

Dedup

`store()` checks for >80% word overlap (Szymkiewicz-Simpson coefficient) in same category + context. If found, updates existing entry -- replaces content with the newer version (facts change), takes max confidence, updates `updated_at`. Context scoping means "deploy process" in `work` won't collide with `personal`.

Fact Conflict & Consolidation

Facts change. "User's project uses React 18" becomes "User's project uses React 19." The system handles this at three levels:

Level 1: Automatic dedup (storage layer)

When `store()` finds >80% word overlap in the same category, it **replaces the content with the newer version** (not the longer one). The newer fact is assumed to be more current. `updated_at` is set; `created_at` is preserved.

```
store(category="fact", content="Project uses React 18")  -> creates
entry
store(category="fact", content="Project uses React 19")  -> 80%
overlap -> replaces content
# Result: one entry, content="Project uses React 19",
updated_at=now
```

Level 2: LLM-driven correction (tool layer)

When the LLM detects a contradiction (e.g., user says "actually we switched to Vue"), it can:

1. Call `recall(query="frontend framework")` -> finds the old fact with its ID
2. Call `update_memory(knowledge_id="abc-123", content="Project uses Vue 3")` -> updates in place
3. Or if it's a complete replacement: `forget` + `remember`

Level 3: Confidence decay (time layer)

Stale facts naturally lose confidence. If "Project uses React 18" hasn't been referenced in 30+ days, its confidence decays. New facts start at 0.5 and grow with use. Outdated facts gradually disappear from the system prompt in favor of actively-used knowledge.

Decay

`apply_confidence_decay()` runs on every startup. Items not used in 30+ days have confidence multiplied by 0.9. The WHERE clause requires both `last_used < cutoff` AND `updated_at < cutoff` to prevent runaway decay on rapid restarts (see Known Corner Cases).

Prune

`prune(days=90)` hard-deletes conversations and tool_history older than 90 days. Knowledge items are self-regulating via confidence decay -- items below 0.1 can be pruned. Conversations are consolidated before the 90-day prune (see Conversation Consolidation).

Hybrid Search Architecture

Search combines two signals via Reciprocal Rank Fusion (RRF):

```
RRF score = 0.6 / (60 + rank_vector) + 0.4 / (60 + rank_bm25)
```

Search Steps

1. Embed query via Lemonade (`nomic-embed-text-v2-moe-GGUF`, 768-dim)
2. FAISS cosine search on normalized embeddings (IndexFlatIP): top-K x 4 candidates (oversample)
3. FTS5 BM25 search: top-K x 4 candidates (oversample)
4. Deduplicate by ID, apply RRF weights, take top-K x 2 candidates
5. Cross-encoder reranking (`cross-encoder/ms-marco-MiniLM-L-6-v2`, ~22MB) on the fused candidates
6. Return final top-K results
7. Bump confidence +0.02 and increment `use_count` on recalled items

FTS5 Behavior

- AND semantics by default. If zero results, automatic OR fallback.
- Query sanitized via `_sanitize_fts5_query()` to strip FTS5 special characters.
- Input capped at 500 chars before regex processing.

Cross-Encoder Reranking

After RRF fusion, a lightweight cross-encoder rescores each candidate by jointly encoding (query, document) pairs. This catches semantic matches that both vector and BM25 underrank individually.

Model: `cross-encoder/ms-marco-MiniLM-L-6-v2` (~22MB, runs on CPU in <50ms for 10 candidates)

Why this matters: RRF fusion combines two independent rankings. The cross-encoder sees query and document together, enabling it to catch fine-grained relevance signals (negation, qualification, context-dependent meaning) that independent encoders miss. Hindsight (91.4% LongMemEval) attributes a significant portion of its retrieval precision to this step.

The cross-encoder model is loaded lazily on first search and cached for the process lifetime. It does not require Lemonade · it runs via the `sentence-transformers` library already in GAIA's dependencies.

Complexity-Aware Recall Depth

Not all queries need the same retrieval depth. Simple factual lookups ("what's my timezone?") need 3 results; complex multi-hop queries ("compare what Sarah and John said about the API migration across recent standups") need thorough retrieval.

The `recall` tool adapts `top_k` based on query complexity:

Complexity	Heuristic signals	top_k	Oversample
Simple	< 8 words, single entity, no comparison words	3	4x
Medium	8-20 words, or contains "how", "why", "explain"	5 (default)	4x
Complex	> 20 words, or contains "compare", "across", "all", "history", "everything"	10	4x

Classification is heuristic (regex on query structure) · no LLM call needed. The oversample factor (4x) is fixed; only the final top_k varies.

```
def _classify_query_complexity(query: str) -> int:
    """Returns adaptive top_k: 3 (simple), 5 (medium), or 10
(complex)."""
    words = query.split()
    complex_signals = {"compare", "across", "all", "history",
"everything", "between", "throughout"}
    medium_signals = {"how", "why", "explain", "describe",
"summarize", "what happened"}

    if len(words) > 20 or complex_signals & set(w.lower() for w in
words):
        return 10
    if len(words) > 8 or medium_signals & set(w.lower() for w in
words):
        return 5
    return 3
```

Hard Requirements

Embedding is not optional. If the Lemonade embedding service is unavailable:

- `init_memory()` raises `RuntimeError("Lemonade embedding service required for memory system")`
- `search_hybrid()` raises `RuntimeError` rather than silently returning BM25-only results
- `store()` raises `RuntimeError` if it cannot embed the new item

This is a deliberate design choice. Silent degradation to keyword-only search produces inconsistent result quality, masks configuration problems, and makes performance benchmarking unreliable. The user should always know when the system is not fully operational.

FAISS Index Lifecycle

- Built on `init_memory()` from stored BLOBs
- Incrementally updated: add on `store()`, remove on `delete()`, replace on `update()`
- Background backfill on startup: embed up to 100 items missing embeddings per init
- Full rebuild via `_rebuild_faiss_index()` (also available as dashboard action)

New MemoryStore Methods (Hybrid Search)

```
store_embedding(knowledge_id: str, embedding: bytes) -> bool
search_hybrid(query_embedding: np.ndarray, query_text: str,
              category=None, context=None, entity=None,
              include_sensitive=False, top_k=5,
              time_from=None, time_to=None) -> List[Dict]
    # time_from/time_to: ISO 8601 strings for temporal filtering
    # Filters on created_at (when the knowledge was first stored)
    # The LLM converts "last week" to concrete dates using current
time from dynamic context
_rebuild_faiss_index() -> None
_get_items_without_embeddings(limit=100) -> List[Dict]
```

New MemoryMixin Methods (Hybrid Search)

```
_get_embedder() -> Any              # Lazy init, cached
LemonadeProvider. Raises RuntimeError if unavailable.
_embed_text(text: str) -> np.ndarray  # Single text -> vector.
Required, not optional.
_backfill_embeddings() -> int       # Embed items missing
embeddings. Called on startup before system is operational.
```

LLM Extraction (Mem0-Inspired)

Knowledge extraction uses a Mem0-inspired pipeline where the LLM sees both the new conversation AND existing memory, then decides what operations to perform. This replaces naive "extract and store" with intelligent memory management · the LLM handles deduplication, contradiction resolution, and knowledge enrichment in a single pass.

Architecture

```
_after_process_query()
        |
```

```
[Fetch top-10 relevant existing items via
search_hybrid(conversation_text)]
        |
[Single LLM call: conversation + existing memory -> operations]
        |
JSON: [{op: "add|update|delete|noop", ...}]
        |
[Execute operations against MemoryStore]
        |
[Embed new/updated items]
```

Why This Matters

Standard extraction pipelines extract new items and rely on string-similarity dedup:

- "User prefers Python" stored · later "I've switched to Rust" · 20% word overlap · BOTH stored
- The system now has contradictory facts with no resolution

Mem0-style extraction shows the LLM what already exists:

- "User prefers Python" in existing memory · "I've switched to Rust" in conversation
- LLM returns: `{op: "update", knowledge_id: "abc-123", content: "User prefers Rust (switched from Python)"}`
- One fact, correctly updated, with lineage preserved via `superseded_by`

Trigger

In `_after_process_query()`, for turns >= 20 words, after conversation storage.

Extraction Prompt

Stored as `_EXTRACTION_PROMPT` constant in `memory.py`:

```
You are a memory manager. Given a conversation turn and the user's
existing memory,
decide what knowledge operations to perform. Return a JSON array
only.
Each item must have an "op" field:
- "add": New knowledge not already in memory
  Required: {op, category, content, entity?, domain?, confidence:
0.4}
- "update": Modify an existing memory item (correction, enrichment,
or supersession)
  Required: {op, knowledge_id, content, entity?, domain?}
- "delete": Remove a memory item contradicted or invalidated by new
information
  Required: {op, knowledge_id, reason}
- "noop": Information already captured accurately. Do not include
in output.
Categories: fact, preference, error, skill, note, reminder
Entity format: type:name (person:sarah_chen, app:vscode,
project:gaia)
Domain examples: journal, meeting, meeting:standup, research,
deployment
```

```
Rules:
- Only extract information useful in FUTURE conversations
- Skip greetings, task confirmations, and ephemeral details
- Prefer "update" over "add" + "delete" when a fact has changed
- Use "delete" only when information is explicitly contradicted
- If nothing worth doing, return []
Existing memory:
{existing_items_json}
Conversation:
User: {user_input}
Assistant: {assistant_response}
```

Integration Point

```
# In _after_process_query(), after storing conversation turns:
if len(user_input.split()) >= 20:  # Skip trivial turns
    # 1. Fetch relevant existing memory for context
    existing = self._memory_store.search_hybrid(
        query_embedding=self._embed_text(user_input),
        query_text=user_input,
        context=self._memory_context,
        top_k=10,
    )

    # 2. LLM decides operations against existing memory
    operations = self._extract_via_llm(user_input,
assistant_response, existing)

    # 3. Execute each operation
    for op in operations:
        if op["op"] == "add":
            self._memory_store.store(
                category=op["category"],
                content=op["content"],
                confidence=op.get("confidence", 0.4),
                entity=op.get("entity"),
                domain=op.get("domain"),
                source="llm_extract",
                context=self._memory_context,
            )
        elif op["op"] == "update":
            # Mark old item as superseded, store new version
            old_id = op["knowledge_id"]
            existing_item = next((e for e in existing if e["id"] ==
old_id), {})
            new_id = self._memory_store.store(
                category=op.get("category",
existing_item.get("category", "fact")),
                content=op["content"],
                confidence=max(existing_item.get("confidence",
0.4), 0.4),
                entity=op.get("entity"),
                domain=op.get("domain"),
```

```
                    source="llm_extract",
                    context=self._memory_context,
                )
                self._memory_store.update(old_id, superseded_by=new_id)
            elif op["op"] == "delete":
                self._memory_store.delete(op["knowledge_id"])
```

Fact Lineage (Zep-Inspired)

When the LLM issues an "update" operation, the old knowledge item is not deleted · it is marked with `superseded_by = new_item_id`. This preserves history:

```
-- Current fact (active):
id="new-456", content="User prefers Rust", superseded_by=NULL
-- Superseded fact (historical):
id="old-123", content="User prefers Python",
superseded_by="new-456"
```

Active queries (`search_hybrid`, `get_by_category`, system prompt injection) filter on `WHERE superseded_by IS NULL` to return only current facts. Historical queries can traverse the `superseded_by` chain to answer "what did I used to prefer?" or "when did this change?"

Error Handling

LLM extraction is a hard requirement when Lemonade is available. If extraction fails:

- **Lemonade unreachable**: `init_memory()` already failed at startup · this state cannot occur at runtime
- **LLM returns invalid JSON**: Log error with full response, skip extraction for this turn (no silent degradation to an inferior method)
- **LLM timeout (3s)**: Log warning, skip extraction for this turn
- **Individual operation fails** (e.g., `knowledge_id` not found for update): Log error, continue with remaining operations

No regex heuristic fallback. If extraction fails, it fails visibly. The LLM still has explicit `remember()` / `update_memory()` / `forget()` tools for anything the auto-extraction misses.

New MemoryMixin Methods

```
_extract_via_llm(user_input: str, assistant_response: str,
                 existing_items: List[Dict]) -> List[Dict]
    """Mem0-style extraction: ADD/UPDATE/DELETE/NOOP against
existing memory. Timeout: 3s."""
_get_embedder() -> Any          # Lazy init, cached
LemonadeProvider
_embed_text(text: str) -> np.ndarray  # Single text -> vector
(required, not optional)
_backfill_embeddings() -> int   # Embed items missing embeddings.
Called on startup.
```

Conversation Consolidation

Purpose

Distill old conversation sessions into durable knowledge before they age out, preventing the 90-day hard delete from destroying accumulated context.

Trigger

- **Automatic:** `init_memory()` on startup -- max 5 sessions per run
- **Manual:** `POST /api/memory/consolidate` REST endpoint

Criteria for Consolidation

- All turns in session are > 14 days old
- Session has >= 5 turns
- At least one turn has `consolidated_at IS NULL`

Consolidation Prompt

```
Summarize this conversation session in 2-3 sentences. Extract any
durable knowledge worth preserving.
Return JSON only: {"summary": "...", "knowledge": [{"category":
"...", "content": "...", "entity": "...or null"}]}
Only extract information useful in future conversations. If nothing
worth extracting, return knowledge as [].
Session ({n} turns, {first_ts} to {last_ts}):
{turns_text}
```

Consolidation Lifecycle

1. Select unconsolidated sessions (query by `consolidated_at IS NULL`, age, turn count)
2. Fetch up to 20 turns per session (oldest first)
3. Call LLM with consolidation prompt
4. Store summary: `knowledge(category="note", source="consolidation", domain="session:{id[:8]}", confidence=0.5)`
5. Store each extracted item via `store()` (normal dedup applies)
6. Mark all fetched turns: `UPDATE conversations SET consolidated_at=now WHERE id IN (...)`
7. Turns remain until 90-day prune; `consolidated_at` prevents re-processing

Storage Impact

Before consolidation	After consolidation
N full conversation turns (up to 4000 chars each)	1 summary note (~500 chars) + M extracted knowledge items
Deleted at 90 days	Summary note: indefinite (subject to confidence decay)
FTS-searchable by exact words	FTS-searchable by summary content AND by extracted knowledge

New MemoryStore Methods

```
get_unconsolidated_sessions(older_than_days=14, min_turns=5,
                            limit=5) -> List[str]   # Returns
session_ids
mark_turns_consolidated(turn_ids: List[int]) -> int  # Returns
```

```
count marked
```

New MemoryMixin Method

```
consolidate_old_sessions(max_sessions=5) -> Dict  # Returns
{consolidated, extracted_items}
```

Background Memory Reconciliation (Hindsight-Inspired)

Over time, knowledge items accumulate from different sources (tool calls, LLM extraction, consolidation, bootstrap) at different times. Items stored weeks apart may reinforce, contradict, or partially overlap each other · but were never co-retrieved during extraction, so conflicts go undetected.

The Problem

In March: `store(category="fact", content="We use PostgreSQL for the API", context="work")` In April: LLM extraction stores `"The team migrated to DynamoDB last quarter"` from an unrelated conversation.

The Mem0-style extraction only sees the current conversation + top-10 existing items. It didn't retrieve the March PostgreSQL fact because the April conversation wasn't about databases. Now two contradictory facts coexist, both active.

Solution: Periodic Reconciliation

On startup, after confidence decay and before consolidation, run a reconciliation pass:

1. **Find high-similarity pairs**: For each context, compute pairwise embedding similarity among active items. Flag pairs with cosine similarity > 0.85.
2. **Classify relationship**: For each flagged pair, a single LLM call classifies the relationship:

   ```
   Given these two memory items from the same user, classify their
   relationship.
   Return JSON: {"relationship":
   "reinforce|contradict|weaken|neutral", "action": "description"}
   Item A (stored {date_a}): {content_a}
   Item B (stored {date_b}): {content_b}
   ```

Relationship	Action
reinforce	Boost confidence of both items by +0.05
contradict	Supersede the older item (`superseded_by = newer_id`), boost newer confidence +0.1
weaken	Reduce confidence of the older item by 0.1 (it may be partially outdated)
neutral	No action (similar words but different topics)

3. **Rate limiting**: Max 20 pair classifications per startup (~20s on local LLM). Process highest-similarity pairs first. Items already reconciled (tracked via `metadata.reconciled_at`) are skipped.

Why This Matters

Without reconciliation:

- Contradictory facts persist indefinitely, confusing the LLM
- The system prompt may inject both "uses PostgreSQL" and "migrated to DynamoDB"
- The user has to manually find and fix conflicts via the dashboard

With reconciliation:

- Contradictions are detected and resolved automatically
- Older facts are superseded with lineage preserved
- Reinforcing facts grow in confidence (used more · more prominent in system prompt)

New MemoryMixin Method

```
def reconcile_memory(self, max_pairs: int = 20) -> Dict:
    """Background reconciliation of high-similarity knowledge
pairs.
    Called on startup after decay, before consolidation.
    Returns: {pairs_checked, reinforced, contradicted, weakened,
neutral}"""
```

Startup Sequence (Updated)

```
init_memory()
  1. Open/create DB, apply schema migrations
  2. Validate Lemonade embedding service connectivity  [HARD
REQUIREMENT]
  3. Backfill embeddings for items missing them
  4. Rebuild FAISS index from stored embeddings
  5. apply_confidence_decay()                          [30-day
decay]
  6. reconcile_memory()
[Hindsight-inspired, max 20 pairs]
  7. consolidate_old_sessions()                        [max 5
sessions]
  8. prune()                                           [90-day hard
delete]
  9. Generate session UUID
```

System Prompt Architecture

The system prompt is split into two parts to allow LLM inference engines to reuse their KV-cache across conversation turns.

Stable Prefix

`get_memory_system_prompt()` -- injected once via `Agent._get_mixin_prompts()`. Contains nothing time-sensitive (no timestamps, no due dates). Stays frozen for the entire session so KV-cache can be reused. Rebuilt only when context changes.

```
def get_memory_system_prompt(self) -> str:
    """Called by Agent._get_mixin_prompts() -- injects STABLE
memory only.
    Injects into the system prompt:
    1. All user preferences in active context (max 10)
    2. Top 5 high-confidence facts in active context
    3. Top 3 high-confidence skills
```

```
    4. Recent error patterns (max 5)
    Deliberately excludes current time and upcoming items -- those
go in
    get_memory_dynamic_context() so this prompt stays frozen for
KV-cache reuse.
    Filters:
    - global context items always included regardless of active
context
    - Sensitive items (sensitive=1) are NEVER included
    """
```

Example stable system prompt fragment:

```
=== MEMORY ===
Preferences:
  - tone: professional but friendly
  - code_style: black formatter, 88 char lines
Known facts:
  - Project uses React 19 with app router (confidence: 0.82)
  - User's name is Alex, role is tech lead (confidence: 0.95)
Skills:
  - Deploy workflow: test -> build -> push -> verify (confidence:
0.88)
  - Docker compose: always use --build flag on first run
(confidence: 0.72)
  - Git bisect: use binary search for regression hunting
(confidence: 0.65)
Known errors to avoid:
  - execute_code: "import torch" fails -- torch not installed on
this machine
  - pip install: always use --index-url for PyTorch packages
```

Dynamic Suffix

`get_memory_dynamic_context()` -- prepended to the user message each turn. Contains current time and upcoming/overdue items. Changes every turn.

```
def get_memory_dynamic_context(self) -> str:
    """Per-turn context injected by process_query() override.
    Contains:
    1. Current date/time (ISO 8601 + day of week)
    2. Upcoming/overdue items (due within 7 days)
    Returns empty string if nothing time-sensitive is active.
    """
```

Example dynamic context prepended to each user message:

```
[GAIA Memory Context]
Current time: 2026-04-01T10:30:00-07:00 (Tuesday)
Upcoming/overdue:
  - [OVERDUE 2026-03-28] Follow up on deployment review
  - [DUE 2026-04-03] Online course starts this week
After mentioning a time-sensitive item, call update_memory to set
reminded_at.
<actual user message here>
```

process_query Override

```
def process_query(self, user_input, **kwargs):
    """Prepend per-turn dynamic context to the user message.
    The system prompt is NOT invalidated -- it stays frozen for
KV-cache reuse.
    The original user_input is saved so _after_process_query can
store the
    clean version (without the dynamic context prefix) to
conversation history.
    """
    self._original_user_input = user_input
    dynamic = self.get_memory_dynamic_context()
    augmented = f"{dynamic}\n\n{user_input}" if dynamic else
user_input
    return super().process_query(augmented, **kwargs)
```

Temporal Query

The temporal query is a simple indexed lookup -- no FTS5 needed:

```
-- Python computes: now_iso =
datetime.now().astimezone().isoformat()
--                  future_iso = (datetime.now().astimezone() +
timedelta(days=7)).isoformat()
-- Passed as parameters (NOT using SQLite datetime() which doesn't
handle timezones)
SELECT * FROM knowledge
WHERE due_at IS NOT NULL
  AND due_at <= ?                                   -- ? =
future_iso (due within a week)
  AND (reminded_at IS NULL OR reminded_at < due_at) -- not reminded
since it became due
ORDER BY due_at ASC
LIMIT ?   -- parameterized, default 10; callers can override via
limit= parameter
```

Agent Hooks

The mixin hooks memory into the Agent lifecycle at exactly **3 points**.

Hook 1: System Prompt Injection + Per-Turn Dynamic Context

Described in the System Prompt Architecture section above. Two methods:

- `get_memory_system_prompt()` -- stable prefix, KV-cache friendly
- `get_memory_dynamic_context()` -- per-turn suffix with current time + upcoming items

Hook 2: Tool Execution Wrapper

```
# Memory tools that should NOT be logged to tool_history (avoids
```

```
noise/recursion)
_MEMORY_TOOLS = {"remember", "recall", "update_memory", "forget",
"search_past_conversations"}
def _execute_tool(self, tool_name: str, tool_args: dict) -> Any:
    """Override Agent._execute_tool() to auto-log every tool call.
    1. Record start time
    2. Call super()._execute_tool()
    3. If tool_name NOT in _MEMORY_TOOLS:
       a. Log to tool_history: name, args, result, success/failure,
duration
       b. If failure and error is novel: auto-store as
knowledge(category='error')
    4. Return original result unchanged
    Memory tools (remember, recall, etc.) are excluded from logging
to avoid
    noise and potential recursion (error auto-store calls remember
internally).
    """
```

Hook 3: Post-Query Conversation Storage + Extraction

```
def _after_process_query(self, user_input: str, assistant_response:
str) -> None:
    """Called after process_query() completes.
    1. Store both turns in conversations table (tagged with active
context)
    2. Mem0-style LLM extraction (for turns >= 20 words):
       - Fetch top-10 relevant existing items via search_hybrid()
       - Call _extract_via_llm() with conversation + existing
memory
       - LLM returns operations: ADD, UPDATE, DELETE, or NOOP
       - Execute operations (store new, supersede old, delete
contradicted)
       - Embed all new/updated items
    No fallback. If extraction fails, it fails visibly (logged
error).
    The LLM still has explicit memory tools for anything
auto-extraction misses.
    """
```

Memory Tools (5 tools, exposed to the LLM)

```
@tool("remember")
def remember(fact: str, category: str = "fact", domain: str = "",
             due_at: str = "", context: str = "", sensitive: str =
"false",
             entity: str = "") -> dict:
    """Store a fact, preference, or learning in persistent memory.
    Categories: fact, preference, error, skill, note, reminder
    If a similar fact already exists (>80% overlap in same
context), it will be updated.
```

```
    Use due_at for time-sensitive items (ISO 8601 format).
    Use context to scope memories (e.g., "work", "personal",
"project-x").
    Use sensitive="true" for private data (excluded from system
prompt).
    Use entity to link to a person/app/service (e.g.,
"person:sarah_chen").
    Examples:
      remember(fact="User prefers concise answers",
category="preference")
      remember(fact="Project uses Next.js 15", category="fact",
domain="frontend",
               context="work")
      remember(fact="Online course starts", category="fact",
               due_at="2026-03-25T09:00:00-07:00")
      remember(fact="Sarah's email is sarah@company.com",
category="fact",
               entity="person:sarah_chen", sensitive="true")
    """
@tool("recall")
def recall(query: str = "", category: str = "", context: str = "",
           entity: str = "", limit: int = 5,
           time_from: str = "", time_to: str = "") -> dict:
    """Search memory for relevant knowledge.
    With query: uses hybrid semantic+keyword search (vector + BM25,
cross-encoder reranking).
    Without query: returns entries filtered by
category/context/entity.
    At least one of query, category, context, entity, or time range
must be provided.
    time_from/time_to: ISO 8601 boundaries. Use current time from
context to compute ranges.
      Example: for "last week", compute Monday 00:00 to Sunday
23:59 of previous week.
    Returns results with IDs, timestamps, and due dates.
    Use ID with update_memory or forget.
    Examples:
      recall(query="deployment process")
      recall(query="user preferences", category="preference")
      recall(entity="person:sarah_chen")             # everything
about Sarah
      recall(query="API migration", time_from="2026-03-01",
time_to="2026-03-31")
      recall(time_from="2026-03-25", time_to="2026-04-01")  #
everything from last week
      recall(category="note", time_from="2026-04-01")        #
today's notes
    """
@tool("update_memory")
def update_memory(knowledge_id: str, content: str = "",
                  category: str = "", domain: str = "",
                  due_at: str = "", reminded_at: str = "",
                  context: str = "", sensitive: str = "",
                  entity: str = "") -> dict:
```

```
    """Update an existing memory entry. Use recall first to find
the ID.
    Only non-empty fields are updated; empty strings are ignored.
    Set reminded_at="now" after mentioning a time-sensitive item to
the user.
    Examples:
      update_memory(knowledge_id="abc-123", content="Project now
uses React 19")
      update_memory(knowledge_id="abc-123", reminded_at="now")  #
mark as mentioned
      update_memory(knowledge_id="abc-123", sensitive="true")   #
mark as sensitive
    """
@tool("forget")
def forget(knowledge_id: str) -> dict:
    """Remove a specific memory entry by ID."""
@tool("search_past_conversations")
def search_past_conversations(query: str = "", days: int = 0,
                              limit: int = 10,
                              time_from: str = "", time_to: str =
"") -> dict:
    """Search past conversation history across all sessions.
    Use query for keyword search, days for time-based retrieval, or
both.
    time_from/time_to: ISO 8601 boundaries for timestamp filtering.
    At least one of query, days, or time range must be provided.
    Examples:
      search_past_conversations(query="database migration")
      search_past_conversations(days=7)  # everything from last
week
      search_past_conversations(query="deploy", days=14)
      search_past_conversations(time_from="2026-03-01",
time_to="2026-03-31")
    """
```

Why 5 tools, not 8? The gaia6 version had 8 overlapping tools. Unified:

- `remember` + `store_insight` + `store_preference` -> `remember` with `category` param
- `recall_memory` + `recall` + `get_preference` -> `recall` with `category` filter
- NEW: `update_memory` -- modify existing entries (recall -> get ID -> update)
- `forget_memory` -> `forget`
- `search_conversations` -> `search_past_conversations`

The CRUD operations map cleanly: `remember` = create, `recall` = read, `update_memory` = update, `forget` = delete. Plus `search_past_conversations` for conversation history.

Temporal Awareness & Wake-up

Temporal Impression

Every knowledge item carries four timestamps: `created_at`, `updated_at`, `last_used`, and optionally `due_at`. Every conversation turn carries `timestamp`. This enables temporal queries without a dedicated temporal

parser.

How temporal search works:

The LLM knows the current time from the dynamic context. When the user asks "what did we discuss last week?", the LLM:

1. Computes the time range (e.g., `2026-03-25T00:00:00-07:00` to `2026-03-31T23:59:59-07:00`)
2. Calls `recall(query="discussed", time_from="2026-03-25T00:00:00-07:00", time_to="2026-03-31T23:59:59-07:00")`
3. Or calls `search_past_conversations(query="...", days=7)`

The `time_from` / `time_to` parameters on `search_hybrid()` add a SQL WHERE clause on `created_at`:

```
AND created_at >= ?  -- time_from
AND created_at <= ?  -- time_to
```

This filters BEFORE vector/BM25 ranking, so only temporally-relevant items enter the fusion pipeline. Combined with hybrid search, this enables queries like "what articles about transformers did I save in March?" · temporal filter narrows to March, semantic search finds transformer-related items.

Temporal awareness in the `recall` tool:

The `recall` tool adds optional `time_from` and `time_to` string parameters (ISO 8601). The LLM is instructed in the tool docstring to convert natural language dates to ISO ranges using the current time from dynamic context.

Core Design

The `due_at` and `reminded_at` fields on the `knowledge` table enable time-sensitive items:

- `due_at` -- when the item becomes actionable (ISO 8601)
- `reminded_at` -- when the agent last surfaced this to the user (ISO 8601)
- `get_upcoming(within_days=7, include_overdue=True)` -- returns items due within N days or overdue

The dynamic suffix (per-turn injection) surfaces upcoming/overdue items. The LLM sets `reminded_at="now"` after mentioning each item.

Wake-up Integration

The wake-up mechanism requires zero memory schema changes. Three integration paths:

1. **Electron tray:** polls `GET /api/memory/upcoming?days=0` every 60 seconds
2. **cron job:** `gaia memory reminders --check --notify`
3. **FastAPI background task:** asyncio loop in `ui/server.py` lifespan

On due items found:

- Fire OS notification AND/OR open new agent session with item as context
- LLM sets `reminded_at="now"` after surfacing each item

Recurring Reminders

LLM advances `due_at` to the next occurrence after firing. No `recurrence_rule` field needed -- LLM handles date arithmetic.

Example: "remind me every Monday" -> LLM sets `due_at` to next Monday after each reminder fires.

```
User: "Remind me to do a weekly review every Friday at 5pm."
-> LLM calls:
  remember(fact="Weekly review every Friday at 5pm",
           category="reminder", due_at="2026-04-04T17:00:00-07:00",
           context="personal", domain="habit:weekly-review")
-> On Friday at 5pm, scheduler surfaces: "[DUE TODAY] Weekly review
every Friday at 5pm"
-> After agent surfaces it, LLM calls:
  update_memory(knowledge_id="...",
                due_at="2026-04-11T17:00:00-07:00",  # advance to
next Friday
                reminded_at="now")
-> Recurrence maintained by the LLM advancing due_at each time
```

Temporal Search

Every knowledge item carries `created_at` and `updated_at` timestamps. The `recall` tool accepts `time_from` and `time_to` parameters (ISO 8601) to filter results by time range. Combined with hybrid search and category filters, this enables queries like:

- "What did I discuss last week?" ·`recall(query="discuss", time_from="2026-03-25", time_to="2026-04-01")`
- "What meetings did I have in March?" ·`recall(category="note", domain="meeting", time_from="2026-03-01", time_to="2026-03-31")`
- "What errors have I seen today?" ·`recall(category="error", time_from="2026-04-01")`

The LLM converts natural language time expressions ("last week", "yesterday", "in March") to concrete ISO 8601 dates using the current time provided in dynamic context. No separate temporal parser is needed · the LLM already knows the current date and can compute offsets.

`search_past_conversations()` also accepts `time_from` and `time_to` for the same temporal filtering on conversation history.

Context Scoping

Different areas of your life produce different knowledge. Without scoping, the system prompt mixes "deploy with `kubectl apply`" (work) with "dentist appointment Thursday" (personal) with "use 4-space indent" (side project). Context keeps them separate.

Context	When it's active	What it contains
`global`	Always included	Universal preferences, name, timezone
`work`	When agent is used for work tasks	Colleagues, project details, work tools
`personal`	Personal assistant mode	Appointments, health goals, personal contacts
`project-x`	User-defined per project	Project-specific facts, skills, errors

How it works:

- `init_memory(context="work")` sets the active context at startup
- `set_memory_context("personal")` switches mid-session
- System prompt includes `global` + active context items
- `remember()` defaults to the active context (overridable per call)
- `recall()` searches across all contexts by default, filterable with `context=`
- Dedup is scoped to context -- "deploy process" in `work` doesn't collide with `personal`

Sensitivity Classification

Some knowledge is private -- email addresses, API tokens, health information, financial data. The `sensitive` flag controls visibility:

| Where | sensitive=0 (default) | sensitive=1 | |---|---|---| | System prompt | Included | Never included | | `recall()` results | Returned | Returned (explicit query) | | Tool history `args` | Full args logged | Args redacted to keys only | | Dashboard | Normal display | Badge, content blurred until clicked |

The LLM can still access sensitive data via `recall()` -- it just won't be broadcast in the system prompt where it could leak into logs or debugging output.

Use Case Mapping

"Remember that my meeting is at 3pm"

```
User -> LLM has memory tools -> calls remember(fact="Meeting at 3pm
today")
-> MemoryStore.store(category="fact", content="Meeting at 3pm
today")
-> Next query: system prompt includes "Meeting at 3pm today"
(high-confidence fact)
```

"I have a course starting next week"

```
Day 1 (March 18):
  User -> LLM calls remember(fact="Online course starts next week",
                             category="fact",
due_at="2026-03-25T09:00:00-07:00")
  -> stored with due_at
Day 5 (March 22):
  User starts a conversation about something unrelated
  -> get_memory_dynamic_context() runs get_upcoming(within_days=7)
  -> Dynamic context prepended to user message: "[DUE Mar 25]
Online course starts next week"
  -> LLM proactively mentions: "By the way, your online course
starts in 3 days"
  -> LLM calls update_memory(knowledge_id="...", reminded_at="now")
  -> Item won't appear in upcoming again (reminded_at is set)
Day 8 (March 25):
  User starts a new session
  -> reminded_at was set, but due_at has now passed
  -> get_upcoming() includes overdue items where reminded_at <
due_at
     (was reminded before it was due, but not after)
  -> Dynamic context: "[TODAY] Online course starts today"
  -> LLM: "Your course starts today! How did it go?"
  -> LLM updates reminded_at again
```

"What did we talk about last week?"

```
User -> LLM knows current time from dynamic context
-> LLM computes time range: Monday to Sunday of previous week
-> Calls recall(query="discussed", time_from="2026-03-24",
time_to="2026-03-30")
   -> Returns knowledge items created during that period, ranked by
hybrid search
-> Or calls search_past_conversations(days=7)
   -> Returns conversation turns from the last 7 days
-> LLM summarizes findings for user
For topic-specific temporal queries ("what did we discuss about
deployment last week?"):
-> recall(query="deployment", time_from="2026-03-24",
time_to="2026-03-30")
   -> Hybrid search (vector+BM25) filtered to the time window
```

Accountability: "I committed to exercising 3x this week"

```
Day 1:
  User -> LLM calls remember(fact="User committed to exercising 3x
this week",
                              category="fact",
due_at="2026-03-23T20:00:00-07:00")
Day 5 (due date):
  User starts any conversation
  -> Dynamic context: "[DUE TODAY] User committed to exercising 3x
this week"
  -> LLM: "Hey, end of the week -- how did the exercise commitment
go? Did you hit 3x?"
  -> Based on user's answer, LLM might:
    - update_memory with outcome in metadata
    - remember a new follow-up for next week
    - forget if no longer relevant
```

Tool fails -> agent learns

```
Agent calls execute_code(code="import torch") -> fails with
ImportError
-> _execute_tool wrapper logs: tool_history(success=0,
error="ImportError: torch")
-> Auto-stores: knowledge(category='error', content="import torch
fails: not installed")
-> Next time: system prompt includes error pattern -> LLM
avoids/handles it
```

User says "I prefer concise answers"

```
User input -> LLM extraction catches preference
-> MemoryStore.store(category="preference", content="User prefers
concise answers",
                     source="llm_extract")
-> Next session: system prompt includes preference -> LLM adjusts
```

```
behavior
```

Agent learns a workflow

```
User walks agent through multi-step deployment 3 times
-> Agent notices pattern -> calls remember(fact="Deploy workflow:
test -> build -> push -> verify",
                                          category="skill",
domain="deployment")
-> Next time user says "deploy": system prompt includes skill ->
LLM follows learned workflow
```

Note-Taking: "Remember that the auth token expires every 24 hours"

```
User -> LLM calls remember(fact="Auth token expires every 24h --
refresh before long jobs",
                            category="note", domain="auth",
context="work")
-> Stored with confidence=0.5
-> Any future query about auth/tokens: system prompt or recall
surfaces this
-> User can view/edit in Memory Dashboard -> Knowledge Browser
```

Journaling: "Log today's work"

```
User: "I finished the memory spec today, reviewed the analysis
docs, and
       pushed the feature branch. Still blocked on the CI lint
step."
-> LLM calls:
  remember(fact="2026-04-01: Completed memory spec, reviewed
analysis docs,
                  pushed feature/agent-memory. Blocked: CI lint.",
            category="note", domain="journal", context="work")
-> Stored as a dated note. Future queries:
  - "What did I work on last Tuesday?" -> recall(query="journal
2026-04-01")
  - "When did I push the memory branch?" ->
recall(query="feature/agent-memory")
  - "What have I been blocked on?" -> recall(query="blocked",
category="note")
```

Meeting Notes: "Capture what was discussed in the standup"

```
User: "In today's standup: Sarah said the API migration is done.
John is blocked
       waiting for design review. Deadline for Q2 report moved to
April 15."
-> LLM calls:
  remember(fact="Standup 2026-04-01: API migration complete
(Sarah). John blocked
                  on design review. Q2 report deadline: April 15.",
```

```
            category="note", domain="meeting:standup",
context="work",
            entity="project:q2-report")
  remember(fact="Q2 report due April 15 -- deadline moved",
            category="reminder", due_at="2026-04-14T09:00:00-07:00",
            context="work", entity="project:q2-report")
  remember(fact="John blocked waiting for design review",
            category="fact", context="work", entity="person:john")
-> Future queries:
  - "What's the Q2 report deadline?" -> recall(query="Q2 report
deadline")
  - "Is John still blocked?" -> recall(entity="person:john")
  - "What did we cover in standups last week?" ->
recall(query="standup", category="note")
```

Personal Knowledge Management: "Save this article summary"

```
User pastes a link or summary about a technical topic.
-> LLM summarizes key points, calls:
  remember(fact="[Source: article title] Key insight: ...",
            category="fact", domain="research", context="personal")
-> Future queries:
  - "What do I know about transformers?" ->
recall(query="transformers", context="personal")
  - "What articles have I read about ML?" ->
recall(category="fact", domain="research")
```

Scheduled Wake-Up Reminder: "Remind me before the Q2 deadline"

```
User: "Remind me two days before the Q2 report deadline."
-> LLM calls:
  remember(fact="Prepare Q2 report for April 15 deadline",
            category="reminder", due_at="2026-04-13T09:00:00-07:00",
            context="work", entity="project:q2-report")
Wake-up path (no agent change needed):
  -> Electron tray / cron calls GET /api/memory/upcoming?days=0
  -> Returns overdue/exactly-due items
  -> Triggers OS notification: "GAIA Reminder: Prepare Q2 report
(due in 2 days)"
  -> Or triggers a new agent session with the reminder as the
opening context
  -> Agent surfaces it: "Heads up -- Q2 report is due in 2 days.
Want to start on it?"
  -> LLM calls update_memory(knowledge_id="...", reminded_at="now")
```

Conversation Consolidation: Long-running agent accumulates months of history

```
After 3 months of daily use, conversations table has ~10,000 turns.
Sessions older than 14 days are consolidated automatically on
startup:
  -> consolidate_old_sessions() finds sessions > 14 days, >= 5
turns, not yet consolidated
```

```
  -> For each session batch (up to 20 turns), calls local LLM:
    "Summarize this session and extract durable knowledge."
    -> Returns: {summary: "...", knowledge: [{category, content,
entity}]}
  -> Stores summary as: knowledge(category="note",
source="consolidation",

domain="session:{session_id[:8]}")
  -> Each extracted knowledge item goes through normal store() with
dedup
  -> Marks source turns as consolidated_at=now (not deleted --
90-day prune still applies)
  -> Old conversations become searchable via consolidated summary
notes
  -> DB growth slows; useful signal is preserved indefinitely as
knowledge
```

Bootstrap: Day-Zero Onboarding

An empty memory is a useless memory. The agent needs to be valuable from the first interaction -- not after weeks of accumulation. Bootstrap solves the cold-start problem through two mechanisms: **conversation** (the agent asks you) and **discovery** (the agent looks around your PC, with your permission).

Design Principles

1. **Consent-first** -- Every discovery source is opt-in. The agent asks before it looks.
2. **Show before store** -- Discovered facts are presented to the user for review before being committed to memory. The user can edit, reject, or reclassify any item.
3. **Progressive** -- Bootstrap doesn't need to happen all at once. The user can do the conversational part now and system discovery later (or never).
4. **Repeatable** -- Bootstrap can be re-run anytime to refresh the agent's understanding. New discoveries don't overwrite user-edited memories (source='user' is preserved).
5. **Private** -- All discovery happens locally. Nothing leaves the machine. Sensitive discoveries are auto-flagged.

Conversational Onboarding

A guided conversation that runs on first launch (or via `gaia memory bootstrap`). The agent asks questions and stores answers as knowledge entries.

```
Agent: "Hi! I'm your GAIA assistant. Let me learn a bit about you
so I can
        be helpful from the start. You can skip any question."
Agent: "What's your name?"
-> remember(fact="User's name is Alex", category="fact",
context="global")
Agent: "What do you do? (e.g., software engineer, student,
designer)"
-> remember(fact="Alex is a senior software engineer",
category="fact", context="global")
Agent: "What are you mainly going to use me for?"
  User: "Work coding, personal task management, and learning"
```

```
-> remember(fact="Primary use cases: work coding, personal tasks,
learning",
           category="fact", context="global")
-> Suggests creating contexts: "work", "personal", "learning"
Agent: "Any preferences for how I communicate? (concise vs
detailed, formal vs casual)"
-> remember(fact="Prefers concise, casual communication",
category="preference",
           context="global")
Agent: "What's your timezone?"
-> remember(fact="Timezone: America/Los_Angeles (PST/PDT)",
category="fact",
           context="global")
Agent: "What tools/languages do you use most for work?"
  User: "Python, TypeScript, VS Code, git"
-> remember(fact="Primary stack: Python, TypeScript",
category="fact",
           context="work", entity="app:vscode")
-> remember(fact="Uses VS Code as primary IDE", category="fact",
           context="work", entity="app:vscode")
```

Implementation: A predefined question flow in `memory.py` -- a method like `run_bootstrap_conversation()` that the agent or CLI can invoke. Not a separate agent -- just a structured conversation using the existing memory tools.

The questions are adaptive -- if the user says "I'm a student," follow-up questions shift to coursework and study habits, not deployment pipelines.

System Discovery

After conversational onboarding, the agent offers to scan the local system. Each source is a separate opt-in with a clear description of what it will access.

```
Agent: "I can learn more about your setup by looking at your
system.
        Each scan is optional -- I'll show you what I find before
saving anything."
  [ ] File system -- Scan common project folders to understand your
projects
  [ ] Browser -- Read bookmarks and recent history to learn your
interests
  [ ] Installed apps -- Check what applications you use
  [ ] Git repos -- Find your projects and understand your tech
stack
  [ ] Email accounts -- Discover your email addresses (not email
content)
  [Start Selected Scans]  [Skip All]
```

Discovery Sources

| Source | What it reads | What it stores | Sensitive? | |---|---|---|---| | **File system** | `~/`, `~/Documents`, `~/Work`, `~/Projects` -- folder names + file extensions only, not file contents | Project names, languages used (by extension), directory structure | No | | **Browser bookmarks** | Chrome/Edge/Firefox bookmark files (JSON/SQLite) | Bookmarked sites -> interests, tools, frequently visited services | Partial -- flag social media, banking | | **Browser history** | Last 30 days of visited URLs (not page content) | Top domains -> interests, workflow patterns, services

used | Yes -- auto-flag all | | **Installed apps** | Windows Apps & Features registry, Start Menu shortcuts | App inventory -> tools, IDEs, communication apps, creative tools | No | | **Git repos** | Walk project folders for `.git/config` -- read remotes, branch names | Project names, languages (by file extensions), remote URLs (GitHub/GitLab) | Partial -- flag private repos | | **Email accounts** | Windows credential store / Thunderbird profiles -- addresses only | Email addresses -> entity creation (`service:gmail`, `service:outlook`) | Yes -- addresses only, not content |

Discovery Flow

```
1. User selects sources -> Agent scans
2. Agent presents findings as a review list:
   "Here's what I found. Review and edit before I save:"
   PROJECTS (from git + file system):
   [x] gaia -- Python/TypeScript, remote: github.com/amd/gaia
[work]
   [x] personal-site -- Next.js, remote: github.com/alex/site
[personal]
   [ ] old-project -- Java (remove? looks inactive)
   TOOLS (from installed apps):
   [x] VS Code, Docker Desktop, Slack, Chrome, Spotify, OBS
   INTERESTS (from bookmarks):
   [x] AI/ML (arxiv.org, huggingface.co)
   [x] Music production (splice.com, ableton.com)
   [!] Banking (chase.com) [auto-flagged sensitive]
   [Save Selected]  [Edit]  [Cancel All]
3. User reviews -> Agent stores approved items as knowledge entries
```

Discovery Implementation

File: `src/gaia/agents/base/discovery.py`

```
class SystemDiscovery:
    """Local system scanner for bootstrap. No agent dependencies.
    Each method returns a list of DiscoveredFact dicts, NOT stored
directly.
    The caller (MemoryMixin.run_bootstrap) presents them for user
review."""
    def scan_file_system(self, paths: List[Path] = None) ->
List[Dict]
        """Walk project directories. Returns project names +
languages."""
    def scan_git_repos(self, paths: List[Path] = None) ->
List[Dict]
        """Find .git directories. Returns repo names, remotes,
languages."""
    def scan_installed_apps(self) -> List[Dict]
        """Read Windows registry/shortcuts. Returns app
inventory."""
    def scan_browser_bookmarks(self) -> List[Dict]
        """Read Chrome/Edge/Firefox bookmark files. Returns
categorized sites."""
    def scan_browser_history(self, days: int = 30) -> List[Dict]
        """Read browser history DBs. Returns top domains. All
flagged sensitive."""
    def scan_email_accounts(self) -> List[Dict]
```

```
        """Read credential store for email addresses. All flagged
sensitive."""
```

Each method returns dicts like:

```
{
    "content": "Project 'gaia' -- Python/TypeScript,
github.com/amd/gaia",
    "category": "fact",
    "context": "work",          # auto-inferred or "unclassified"
    "entity": "project:gaia",
    "sensitive": False,
    "confidence": 0.4,          # discovery confidence (lower than
user-stated)
    "source": "discovery",
    "approved": None,           # set by user review: True/False
}
```

Auto-Classification

Discovery results are auto-classified into contexts using simple heuristics:

| Signal | Inferred context | |---|---| | Found in `~/Work/` or has corporate git remote | `work` | | Found in `~/Documents/Personal/` or personal domain | `personal` | | Browser bookmark in "Work" folder | `work` | | Social media / entertainment sites | `personal` | | Can't determine | `unclassified` -> user assigns during review |

CLI Integration

```
gaia memory bootstrap              # Run full bootstrap
(conversation + discovery)
gaia memory bootstrap --chat-only  # Conversational onboarding only
gaia memory bootstrap --discover   # System discovery only
(re-scannable)
gaia memory bootstrap --reset      # Clear source='discovery' items
(with confirmation prompt)
gaia memory status                 # Show memory stats (count by
source, category, context)
```

Reset safety: `--reset` only deletes items where `source='discovery'`. Items the user has manually edited via dashboard (source changes to `'user'`) are preserved. Always prompts for confirmation with a count: "Delete 34 discovered items? (y/n)"

Agent UI Integration

A "Setup" or "Get Started" page shown on first launch:

1. Welcome screen explaining what bootstrap does
2. Conversational onboarding (chat interface)
3. Discovery source selection (checkboxes)
4. Review screen (approve/reject/edit findings)
5. Summary ("I learned 47 things about you. Ready to help!")

Accessible anytime from dashboard via "Re-run Bootstrap" button.

Privacy Safeguards

- **No file contents** -- File system scan reads names and extensions only
- **No email content** -- Only discovers email addresses exist
- **No browser page content** -- Only URLs/domains from history
- **No network** -- Everything runs locally, nothing transmitted
- **Auto-flag sensitive** -- Browser history, email, banking sites -> `sensitive=1`
- **User review required** -- Nothing stored without explicit approval
- **Deletable** -- User can delete any bootstrap-discovered item from dashboard
- **Source tracking** -- All bootstrap items tagged `source='discovery'` so user can filter/bulk-delete

---

Dashboard & Observability

Memory is only trustworthy if you can see it. The agent UI has a **Memory Dashboard** -- a window into everything the agent knows, how it's performing, and what's coming up.

MemoryStore Query API

The `MemoryStore` class exposes read-only aggregate methods for the dashboard. No new tables -- these are queries over existing data.

```
class MemoryStore:
    # ... (existing methods) ...
    # --- Dashboard / Observability ---
    def get_stats(self) -> Dict:
        """Aggregate statistics across all tables.
        Returns:
            {
                "knowledge": {
                    "total": 142,
                    "by_category": {"fact": 68, "preference": 12,
"error": 35,
                                    "skill": 27, "note": 5,
"reminder": 3},
                    "by_context": {"global": 15, "work": 95,
"personal": 32},
                    "sensitive_count": 8,
                    "entity_count": 12,        -- unique entities
                    "avg_confidence": 0.64,
                    "embedding_count": 130,    -- items with
embeddings
                    "oldest": "2026-01-15T...",
                    "newest": "2026-04-01T...",
                },
                "conversations": {
                    "total_turns": 1847,
                    "total_sessions": 93,
                    "consolidated_sessions": 42,  -- sessions with
consolidated turns
                    "first_session": "2026-01-15T...",
                    "last_session": "2026-04-01T...",
                },
```

```
                "tools": {
                    "total_calls": 523,
                    "unique_tools": 18,
                    "overall_success_rate": 0.91,
                    "total_errors": 47,
                },
                "temporal": {
                    "upcoming_count": 3,      -- due within 7 days,
not reminded
                    "overdue_count": 1,        -- past due, not
resolved
                },
                "db_size_bytes": 2457600,
            }
        """
    def get_all_knowledge(self, category: str = None,
                          context: str = None,
                          entity: str = None,
                          sensitive: bool = None,
                          search: str = None,
                          sort_by: str = "updated_at",
                          order: str = "desc",
                          offset: int = 0,
                          limit: int = 50) -> Dict:
        """Paginated knowledge browser with full filtering.
        search: optional FTS5 query to filter by content.
        Returns: {"items": [...], "total": 142, "offset": 0,
"limit": 50}"""
    def get_tool_summary(self) -> List[Dict]:
        """Per-tool stats for the tool activity table.
        Returns list of:
            {
                "tool_name": "execute_code",
                "total_calls": 87,
                "success_count": 79,
                "failure_count": 8,
                "success_rate": 0.91,
                "avg_duration_ms": 1230,
                "last_used": "2026-04-01T14:30:00-07:00",
                "last_error": "SyntaxError: unexpected indent",
            }
        """
    def get_activity_timeline(self, days: int = 30) -> List[Dict]:
        """Daily activity counts for the activity chart.
        Returns list of:
            {
                "date": "2026-04-01",
                "conversations": 12,   -- turns that day
                "tool_calls": 8,
                "knowledge_added": 3,
                "errors": 1,
            }
        """
    def get_recent_errors(self, limit: int = 20) -> List[Dict]:
```

```
        """Recent tool errors for the error log view.
        Returns tool_history rows where success=0, newest first."""
```

Dashboard REST API

New FastAPI router following the existing pattern in `src/gaia/ui/routers/`:

File: `src/gaia/ui/routers/memory.py`

```
router = APIRouter(tags=["memory"])
# --- Dashboard ---
@router.get("/api/memory/stats")
async def memory_stats() -> Dict:
    """Aggregate stats for dashboard header cards."""
@router.get("/api/memory/activity")
async def memory_activity(days: int = 30) -> List[Dict]:
    """Daily activity timeline for the activity chart."""
# --- Knowledge Browser ---
@router.get("/api/memory/knowledge")
async def list_knowledge(category: str = None,
                         context: str = None,
                         entity: str = None,
                         sensitive: bool = None,
                         search: str = None,
                         sort_by: str = "updated_at",
                         order: str = "desc",
                         offset: int = 0,
                         limit: int = 50) -> Dict:
    """Paginated, filterable, searchable knowledge entries."""
@router.post("/api/memory/knowledge")
async def create_knowledge(body: KnowledgeCreate) -> Dict:
    """Create a knowledge entry from the dashboard (source='user',
confidence=0.8)."""
@router.put("/api/memory/knowledge/{knowledge_id}")
async def edit_knowledge(knowledge_id: str, body: KnowledgeUpdate)
-> Dict:
    """Edit a knowledge entry from the dashboard (all fields
editable)."""
@router.delete("/api/memory/knowledge/{knowledge_id}")
async def delete_knowledge(knowledge_id: str) -> Dict:
    """Delete a knowledge entry from the dashboard."""
# --- Entities ---
@router.get("/api/memory/entities")
async def list_entities() -> List[Dict]:
    """List all unique entities with their knowledge counts.
    Returns: [{"entity": "person:sarah_chen", "count": 5,
"last_updated": "..."}]"""
@router.get("/api/memory/entities/{entity}")
async def get_entity(entity: str) -> List[Dict]:
    """Get all knowledge linked to a specific entity."""
# --- Contexts ---
@router.get("/api/memory/contexts")
async def list_contexts() -> List[Dict]:
    """List all contexts with their knowledge counts.
```

```
    Returns: [{"context": "work", "count": 68}, {"context":
"personal", "count": 23}]"""
# --- Tool Performance ---
@router.get("/api/memory/tools")
async def tool_summary() -> List[Dict]:
    """Per-tool performance stats."""
@router.get("/api/memory/tools/{tool_name}/history")
async def tool_history(tool_name: str, limit: int = 50) ->
List[Dict]:
    """Recent call history for a specific tool."""
# --- Errors ---
@router.get("/api/memory/errors")
async def recent_errors(limit: int = 20) -> List[Dict]:
    """Recent tool errors across all tools."""
# --- Conversations ---
@router.get("/api/memory/conversations")
async def list_sessions(limit: int = 20) -> List[Dict]:
    """List conversation sessions with timestamps and turn
counts."""
@router.get("/api/memory/conversations/{session_id}")
async def get_session(session_id: str) -> List[Dict]:
    """Get all turns for a specific conversation session."""
@router.get("/api/memory/conversations/search")
async def search_conversations(query: str, limit: int = 20) ->
List[Dict]:
    """Full-text search across all conversations."""
# --- Temporal ---
@router.get("/api/memory/upcoming")
async def upcoming_items(days: int = 7) -> List[Dict]:
    """Time-sensitive items due within N days + overdue."""
# --- Maintenance ---
@router.post("/api/memory/consolidate")
async def consolidate_sessions() -> Dict:
    """Manual trigger for conversation consolidation. Returns
{consolidated, extracted_items}."""
@router.post("/api/memory/rebuild-embeddings")
async def rebuild_embeddings() -> Dict:
    """Rebuild FAISS index and backfill missing embeddings."""
@router.post("/api/memory/rebuild-fts")
async def rebuild_fts() -> Dict:
    """Rebuild FTS5 indexes (recovery from corruption)."""
```

Dashboard UI

File: `src/gaia/apps/webui/src/pages/MemoryDashboard.tsx` (new page)

The dashboard has 6 sections:

1. Header Cards (at-a-glance stats)

```
+--------------+ +--------------+ +--------------+ +--------------+
|  142         | |  93          | |  523         | |  91%
|
|  Memories    | |  Sessions    | |  Tool Calls  | |  Success
```

```
Rate|
|  +3 today     | |  since Jan   | |  18 tools    | |  47 errors
|
+--------------+ +--------------+ +--------------+ +--------------+
```

2. Activity Timeline (contribution graph)

A heatmap or bar chart showing daily activity over the last 30 days:

- Conversations (blue)
- Tool calls (green)
- Knowledge added (purple)
- Errors (red)

Similar to GitHub's contribution graph -- at a glance you see when the agent was most active.

3. Knowledge Browser (main table)

Filterable, sortable table of all knowledge entries:

```
  [Context: All v]  [Category: All v]  [Entity: All v]  [Search:
________]

+---------+---------------------------------------+---------+--------+----------+---
|Category | Content                               | Context |
Entity |Confidence| Due     | Updated  |

+---------+---------------------------------------+---------+--------+----------+---
| fact    | Project uses React 19                 | work    | --
  | 0.82     | --      | Apr 01   |
| pref    | User prefers concise answers          | global  | --
  | 0.65     | --      | Mar 15   |
| error   | import torch fails: not installed     | work    | --
  | 0.70     | --      | Mar 17   |
| fact    | [S] Sarah's email is sarah@company.com | work    |
sarah  | 0.50     | --      | Apr 01   |
| fact    | Online course starts next week        |personal | --
  | 0.50     | Mar 25  | Mar 18   |
| skill   | Deploy: test -> build -> push -> verify | work    | --
   | 0.88     | --      | Mar 16   |

+---------+---------------------------------------+---------+--------+----------+---
  + Add Memory
Page 1 of 3  [< >]
```

- `[S]` = sensitive entry (content blurred until clicked)
- Entity column links to entity profile (click to see all knowledge about this entity)
- Each row clickable for full detail view (metadata, timestamps, source, use_count)
- Inline actions: **Edit** (all fields), **Delete**, **Copy ID**, **Toggle Sensitive**
- + **Add Memory** button creates entries with `source='user'`, `confidence=0.8`

All memory is user-editable through the dashboard. The user is the ultimate authority over what the agent knows -- they can create, correct, update, or delete any entry.

4. Tool Performance (stats table)

```
+----------------+-------+---------+-----------+-----------------------------+
| Tool           | Calls | Success | Avg Time  | Last Error
          |

+----------------+-------+---------+-----------+-----------------------------+
| execute_code   |  87   |  91%    |  1.2s     | SyntaxError:
unexpected...   |
| read_file      |  156  |  98%    |  45ms     | FileNotFoundError:
/tmp/...  |
| web_search     |  43   |  86%    |  2.1s     | ConnectionTimeout:
...       |
| write_file     |  28   |  100%   |  18ms     | --
          |

+----------------+-------+---------+-----------+-----------------------------+
```

Click a tool row to see its full call history.

5. Conversation History Browser

List of past sessions with timestamps, turn counts, and preview of first message. Consolidation status shown per session. Click a session to see full conversation. Search bar for FTS5 across all conversations. Read-only -- conversations are an immutable log (no edit/delete).

6. Upcoming & Overdue (temporal sidebar)

```
+-- Upcoming ----------------------------------------+
| [clock] Apr 03  Online course starts              |
| [clock] Apr 05  Team standup presentation          |
|                                                    |
| [!] OVERDUE                                        |
| [!] Mar 28  Follow up on deployment review         |
+----------------------------------------------------+
```

Dashboard Design Principles

1. **Read-heavy, write-light** -- Dashboard is mostly reading. Writes only for manual edits/deletes.
2. **No real-time streaming needed** -- Data refreshes on page load or manual refresh. No WebSocket needed.
3. **Same DB file** -- The dashboard reads directly from `~/.gaia/memory.db`. No separate data store.
4. **API-first** -- All data flows through the REST API. The frontend never touches SQLite directly.
5. **Pagination everywhere** -- Knowledge and tool history can grow large. Always paginated.

Navigation

Add a "Memory" tab to the agent UI sidebar/nav, alongside existing Chat/Documents tabs.

Class Interfaces

MemoryStore

```
class MemoryStore:
    """Pure SQLite storage for agent memory. No agent
dependencies."""
    def __init__(self, db_path: Path = None):
        """Open/create DB at db_path. Default: ~/.gaia/memory.db
        Uses WAL mode. Thread-safe via threading.Lock.
        Runs schema migrations if needed."""
    # --- Conversations ---
    def store_turn(self, session_id: str, role: str, content: str,
                   context: str = "global") -> None
    def get_history(self, session_id: str = None, context: str =
None,
                    limit: int = 20) -> List[Dict]
    def search_conversations(self, query: str, context: str = None,
                             limit: int = 10) -> List[Dict]
        """FTS5 keyword search across conversation content.
        Filterable by context. For time-filtered queries, use
        get_recent_conversations(days=N) instead."""
    def get_recent_conversations(self, days: int = 7, context: str
= None,
                                 limit: int = 50) -> List[Dict]
        """Get conversations from the last N days (timestamp-based,
not FTS5).
        Filterable by context. Returns turns ordered
oldest-first."""
    def get_sessions(self, limit: int = 20) -> List[Dict]
        """List conversation sessions with turn counts, timestamps,
and first message preview."""
    # --- Knowledge ---
    def store(self, category: str, content: str,
              domain: str = None, metadata: dict = None,
              confidence: float = 0.5,
              due_at: str = None,
              source: str = "tool",
              context: str = "global",
              sensitive: bool = False,
              entity: str = None) -> str
        """Store with dedup: >80% word overlap in same
category+context -> update existing.
        Dedup is scoped to context -- 'work' facts don't collide
with 'personal' facts.
        Validates due_at is a valid ISO 8601 string if provided.
        Confidence clamped to [0.0, 1.0].
        Empty-string entity/domain normalized to None."""
    def search(self, query: str, category: str = None,
               context: str = None, entity: str = None,
               include_sensitive: bool = False,
               top_k: int = 5,
               time_from: str = None,
               time_to: str = None) -> List[Dict]
        """FTS5 BM25 keyword search (the lexical component of
hybrid search).
```

```
        MemoryMixin orchestrates hybrid search by calling both
search() and
        search_hybrid() then fusing results via RRF + cross-encoder
reranking.
        Bumps confidence +0.02 and increments use_count on recalled
items.
        Filters on superseded_by IS NULL (active items only).
        Filters by context/entity if provided. Excludes sensitive
by default.
        time_from/time_to: ISO 8601 boundaries for created_at
filtering.
        Returns dicts with all fields."""
    def get_by_category(self, category: str, context: str = None,
                        limit: int = 10) -> List[Dict]
    def get_by_entity(self, entity: str, limit: int = 20) ->
List[Dict]
        """Get all knowledge about a specific entity.
        Example: get_by_entity('person:sarah_chen') -> all facts
about Sarah."""
    def get_upcoming(self, within_days: int = 7, include_overdue:
bool = True,
                     context: str = None, limit: int = 10) ->
List[Dict]
        """Get time-sensitive items due within N days (or overdue).
        Returns items where either: (a) never reminded, or (b)
reminded before
        the due date but due date has now passed (needs follow-up).
        Filterable by context."""
    def update(self, knowledge_id: str, content: str = None,
               category: str = None, domain: str = None,
               metadata: dict = None, context: str = None,
               sensitive: bool = None, entity: str = None,
               due_at: str = None, reminded_at: str = None,
               superseded_by: str = None) -> bool
        """Update an existing knowledge entry. Only provided fields
are changed.
        Sets updated_at to now. Returns False if ID not found.
        Normalizes reminded_at and due_at to tz-aware ISO 8601.
        When superseded_by is set, marks this item as replaced by a
newer item."""
    def update_confidence(self, knowledge_id: str, delta: float) ->
None
    def delete(self, knowledge_id: str) -> bool
    # --- Embeddings ---
    def store_embedding(self, knowledge_id: str, embedding: bytes)
-> bool
    def search_hybrid(self, query_embedding: np.ndarray,
query_text: str,
                      category=None, context=None, entity=None,
                      include_sensitive=False, top_k=5,
                      time_from: str = None, time_to: str = None)
-> List[Dict]
    def _rebuild_faiss_index(self) -> None
    def _get_items_without_embeddings(self, limit=100) ->
```

```
List[Dict]
    # --- Consolidation ---
    def get_unconsolidated_sessions(self, older_than_days=14,
min_turns=5,
                                    limit=5) -> List[str]
    def mark_turns_consolidated(self, turn_ids: List[int]) -> int
    # --- Tool History ---
    def log_tool_call(self, session_id: str, tool_name: str,
                      args: dict, result_summary: str,
                      success: bool, error: str = None,
                      duration_ms: int = None) -> None
    def get_tool_errors(self, tool_name: str = None,
                        limit: int = 10) -> List[Dict]
    def get_tool_stats(self, tool_name: str) -> Dict
        """Returns: {total_calls, success_rate, avg_duration_ms,
last_error}"""
    # --- Dashboard / Observability ---
    def get_stats(self) -> Dict
    def get_all_knowledge(self, ...) -> Dict
    def get_tool_summary(self) -> List[Dict]
    def get_activity_timeline(self, days: int = 30) -> List[Dict]
    def get_recent_errors(self, limit: int = 20) -> List[Dict]
    def get_entities(self, limit: int = 100) -> List[Dict]
        """List unique entities with knowledge counts."""
    def get_contexts(self, limit: int = 100) -> List[Dict]
        """List contexts with knowledge counts."""
    # --- Housekeeping ---
    def apply_confidence_decay(self, days_threshold: int = 30,
                               decay_factor: float = 0.9) -> int
        """Decay confidence for items not used in N days. Called
once per session start."""
    def prune(self, days: int = 90) -> int
        """Hard-delete conversations and tool_history older than N
days."""
    def rebuild_fts(self) -> None
        """Rebuild FTS5 indexes from source tables."""
    def close(self) -> None
```

Key Behaviors

- **Deduplication:** `store()` checks for >80% word overlap (Szymkiewicz-Simpson coefficient) in same category + context. If found, updates existing entry -- replaces content with the newer version, takes max confidence, updates `updated_at`.
- **Hybrid search:** Vector (FAISS cosine) + BM25 (FTS5) fused via RRF. Both components always run · no degradation to keyword-only. Active items only (`superseded_by IS NULL`). FTS5 uses AND semantics with OR expansion on zero results. Query sanitized; input capped at 500 chars.
- **Confidence:** 0.0-1.0 scale. +0.02 on recall with use_count increment (atomic SQL-side arithmetic), decays x0.9 for items unused >30 days. Clamped to [0.0, 1.0] on storage.
- **Thread safety:** All DB operations protected by `threading.Lock`.
- **Timestamps:** All timestamps use `datetime.now().astimezone().isoformat()` -- local time with UTC offset.
- **Content validation:** Empty/whitespace-only content raises `ValueError`. Content truncated at 2000 chars (knowledge) or 4000 chars (conversations). `due_at` normalized to tz-aware ISO 8601.

- **Rollback discipline:** Every write path wraps DML + `commit()` in `try/except Exception: self._conn.rollback(); raise`.

MemoryMixin

```
class MemoryMixin:
    """Mixin that gives any Agent persistent memory.
    Usage:
        class MyAgent(MemoryMixin, Agent):   # MemoryMixin MUST
come before Agent
            def __init__(self, **kwargs):
                self.init_memory()          # Before
super().__init__()
                super().__init__(**kwargs)
            def _register_tools(self):
                super()._register_tools()
                self.register_memory_tools()
    """
    def init_memory(self, db_path: Path = None, context: str =
"global") -> None
        """Initialize memory store with an active context scope.
        Validates Lemonade connectivity, backfills embeddings,
rebuilds FAISS index, runs confidence decay, memory reconciliation,
session consolidation, and pruning (in that order)."""
    @property
    def memory_store(self) -> MemoryStore
    @property
    def memory_session_id(self) -> str
    @property
    def memory_context(self) -> str
        """Current active context (e.g., 'work', 'personal',
'global')."""
    def set_memory_context(self, context: str) -> None
        """Switch active context. Rebuilds system prompt for
immediate effect.
        Empty/whitespace strings normalized to 'global'."""
    # Prompt integration
    def get_memory_system_prompt(self) -> str
    def get_memory_dynamic_context(self) -> str
    # Tool registration
    def register_memory_tools(self) -> None
    # Lifecycle hooks (overrides Agent methods)
    def process_query(self, user_input, **kwargs) -> Dict
        """Saves original input, prepends per-turn dynamic context,
calls super()."""
    def _execute_tool(self, tool_name, tool_args) -> Any
    def _after_process_query(self, user_input, response) -> None
    # LLM extraction
    def _extract_via_llm(self, user_input: str, assistant_response:
str,
                         existing_items: List[Dict]) -> List[Dict]
        """Mem0-style extraction: ADD/UPDATE/DELETE/NOOP against
existing memory. Timeout: 3s."""
```

```
    # Hybrid search (required -- raises RuntimeError if Lemonade
unavailable)
    def _get_embedder(self) -> Any
    def _embed_text(self, text: str) -> np.ndarray
    def _backfill_embeddings(self) -> int
    # Consolidation
    def consolidate_old_sessions(self, max_sessions=5) -> Dict
    # Session management
    def reset_memory_session(self) -> None
```

Integration with Agent Base Class

Two small changes to `agent.py` (the `process_query` override lives in the mixin, not in agent.py):

Change 1: Mixin prompt check (4 lines)

In `_get_mixin_prompts()`, add after the VLM check:

```
# Check for Memory mixin prompts
if hasattr(self, "get_memory_system_prompt"):
    fragment = self.get_memory_system_prompt()
    if fragment:
        prompts.append(fragment)
```

Change 2: Post-query hook (2 lines)

At the end of `process_query()`, just before `return result`:

```
if hasattr(self, '_after_process_query'):
    self._after_process_query(user_input, result.get("result", ""))
```

Design Constraints

What this system deliberately does not do:

- **No pre-query forced search** -- LLM calls `recall` when it wants (faster, simpler)
- **No multi-user support** -- single user per machine
- **No credential encryption** -- credentials belong in OS keyring, not memory.db
- **No in-agent recurrence rules** -- LLM advances `due_at` per firing; RRULE parsing is a future concern
- **No real-time memory streaming** -- dashboard polls REST; no WebSocket needed
- **No knowledge graph traversal** -- `entity` tags enable basic entity profiling; full graph (typed relations, neighbor queries) is a future layer
- **No ColBERT/late-interaction retrieval** -- single-vector embeddings + BM25 hybrid is sufficient for personal scale; ColBERT adds significant complexity
- **No cloud sync** -- all data stays in `~/.gaia/memory.db`
- **No silent fallback** -- if embeddings or LLM extraction are unavailable, the system fails loudly rather than silently degrading to inferior methods. Consistent behavior is more valuable than maximum availability.
- **No encryption at rest (v2)** -- memory.db is stored in plaintext. AES-256-GCM encryption at rest is a future requirement for multi-user accounts and enterprise deployments. Tracked for v3.

- **No multi-modal memory (v2)** -- knowledge items are text-only. Future versions will support images (whiteboard photos, screenshots, diagrams) and audio (voice memos) as first-class memory items with multi-modal embeddings.

Risks & Mitigations

System Prompt Bloat

As knowledge accumulates, `get_memory_system_prompt()` could grow unbounded and eat into context window budget. **Mitigation:** Hard limits on each section -- max 10 preferences, max 5 facts, max 3 skills, max 5 errors. Total memory prompt section capped at ~2000 tokens. If over budget, prioritize by confidence score.

Database Growth

Conversations and tool_history grow indefinitely. After months of use, the DB could become large. **Mitigation:**

- `tool_history`: Retention policy -- keep last 90 days, prune older entries. `MemoryStore.prune(days=90)`.
- `conversations`: Same retention policy. Consolidation distills old sessions into knowledge entries before pruning.
- `knowledge`: Self-regulating via confidence decay -- stale items drop below 0.1 and can be pruned.
- WAL mode checkpoint: Periodic `PRAGMA wal_checkpoint(TRUNCATE)` to prevent WAL file growth.

Schema Migration

When we add columns or tables in future versions, existing `memory.db` files need migration. **Mitigation:** `schema_version` table (single row) with migration function in `MemoryStore.__init__()`. Check version, apply ALTER TABLE/CREATE TABLE as needed.

FTS5 Index Corruption

FTS5 standalone tables can get out of sync if the process crashes mid-write. **Mitigation:** Use `INSERT OR REPLACE` patterns carefully, and add `rebuild_fts()` method callable from the dashboard if search results seem wrong.

MRO and Multiple Mixins

`MemoryMixin` overrides `process_query` and `_execute_tool` -- both of which are also defined in `Agent`. For MemoryMixin's versions to run first (and call `super()` to reach Agent), **MemoryMixin must appear before Agent in the class declaration**: `class MyAgent(MemoryMixin, Agent, OtherMixin)`. If Agent is listed first, both overrides are silently shadowed and tool logging + dynamic context injection will not work.

LLM Not Using Memory Tools

Local LLMs (especially smaller ones) may not reliably call memory tools -- they might ignore the tools or use wrong arguments. **Mitigation:** The automatic hooks (tool logging, conversation storage, system prompt injection, LLM extraction) work regardless of whether the LLM calls memory tools. The tools are a bonus for smarter models, not a requirement.

Invalid Dates from LLM

The LLM might pass "next Tuesday" or "2026-13-45" as `due_at`. **Mitigation:** The `remember` tool validates `due_at` with `datetime.fromisoformat()` before calling `store()`. If invalid, return an error message telling the LLM to use ISO 8601 format. The current time is in the system prompt so the LLM can compute dates.

Dashboard DB Access

The UI backend (FastAPI) needs its own MemoryStore instance to access `~/.gaia/memory.db`. This is safe because WAL mode supports concurrent readers. **Implementation:** The dashboard uses a single shared read-write `MemoryStore` singleton (thread-safe initialization via `threading.Lock`), which handles both read and write endpoints through the public `MemoryStore` API. Raw SQL access from the router layer is prohibited -- all queries must go through named `MemoryStore` methods. The singleton is lazy-initialized on first request and persists for the lifetime of the server process.

Sync handlers in threadpool: All FastAPI route handlers are defined as `def` (not `async def`) so FastAPI automatically runs them in a worker thread pool. This prevents the SQLite calls from blocking the asyncio event loop. In particular, `get_stats()` issues ~12 sequential queries and `rebuild_fts()` can be slow -- running these synchronously in `async def` handlers would stall all other requests.

Known Corner Cases & Fixes

This section documents reliability issues identified and fixed during iterative hardening. Each entry explains the root cause and the applied fix.

1. Python sqlite3 Implicit Transaction Model

Root cause: Python's `sqlite3` module opens implicit transactions on any DML (`INSERT`/`UPDATE`/`DELETE`). If an exception occurs between the first DML and the `commit()`, the pending changes are not discarded -- they stay in a half-open transaction and will be committed by the *next* unrelated `commit()` on the same connection.

Pattern: Every write path wraps DML + `commit()` in `try/except Exception: self._conn.rollback(); raise`.

Affected methods (all fixed): `store()` (both insert and dedup-update branches), `update()`, `delete()`, `store_turn()`, `log_tool_call()`, `search()` (confidence bump loop), `apply_confidence_decay()`, `update_confidence()`, `rebuild_fts()`, `prune()`.

2. Confidence Decay Idempotency

Root cause: `apply_confidence_decay()` sets `updated_at = now` after decay. Without guarding, a second call within the same decay window (e.g., on rapid restart) would find items with `last_used < cutoff` but `updated_at = now` (just set) and decay them again. After ~22 rapid restarts, a confidence-0.5 item would drop below the 0.1 pruning threshold.

Fix: Added `AND updated_at < cutoff` to the WHERE clause so items decayed in the current period (whose `updated_at` was just bumped to now) are not re-decayed until the next period.

3. `get_sessions()` First Message Ordering

Root cause: `MIN(CASE WHEN role='user' THEN content END)` returns the lexicographically *smallest* user message, not the *chronologically first* one.

Fix: Replaced with a correlated subquery: `(SELECT content FROM conversations c2 WHERE c2.session_id = conversations.session_id AND c2.role = 'user' ORDER BY c2.id ASC LIMIT 1)`.

4. `_extract_heuristics()` Only Found First Match (Historical -- replaced by LLM extraction in v2)

Root cause: Used `pattern.search()` which returns only the first match per pattern, silently dropping subsequent matches in the same message.

Fix: Changed to `pattern.finditer()` to capture all non-overlapping matches per pattern.

5. `log_tool_call()` Unbounded args_json

Root cause: `result_summary` and `error` columns were truncated to 500 chars, but `args_json` (JSON-serialized tool arguments) was stored at full length. A single `write_file` call with large content could store megabytes.

Fix: Added truncation of `args_json` to 500 chars, matching `result_summary` and `error`.

6. WAL Checkpoint Race Condition

Root cause: `PRAGMA wal_checkpoint(TRUNCATE)` was called outside `self._lock`, making it possible for a concurrent `self._conn.execute()` from another thread to race with the checkpoint.

Fix: Moved the checkpoint inside the `with self._lock:` block in `prune()`, wrapped in its own `try/except` (best-effort -- `SQLITE_BUSY` from a reader holding a snapshot is non-fatal).

7. `remember()` Tool Content Not Actually Truncated

Root cause: The `remember()` tool set `was_truncated = len(fact) > 2000` and appended a truncation note to the return message, but passed the full `fact` string (not `fact[:2000]`) to `store()`.

Fix: Changed `content=fact` to `content=fact[:2000]` in the `store()` call.

8. `update_memory()` Tool Content Not Actually Truncated

Root cause: Same pattern as `remember()`. `kwargs["content"] = content` stored the full string.

Fix: Changed `kwargs["content"] = content` to `kwargs["content"] = content[:2000]`.

9. Content Validation Edge Cases

`store_turn()` **empty content:** Empty strings and whitespace-only turns were silently stored, polluting conversation history and the FTS5 index. **Fix:** `store_turn()` returns early when content is empty or whitespace-only.

`store()` **confidence clamping:** Programmatic callers could pass `confidence=999.0` or `confidence=-1.0`, corrupting stats. **Fix:** Confidence clamped to `[0.0, 1.0]` via `max(0.0, min(1.0, float(confidence)))`.

`set_memory_context()` **empty/whitespace:** Passing `""` or `"  "` would set context to empty string, matching nothing. **Fix:** Normalized to `"global"`.

`store()` / `update()` **empty-string entity/domain:** SQLite treats `entity = ""` differently from `entity IS NULL`, breaking dedup and partial indexes. **Fix:** `store()` normalizes `entity=""` and `domain=""` to `None`. `update()` also normalizes these to `None` (no-op under "None = don't change" semantics).

`remember` **tool empty** `fact`**:** Would raise `ValueError` that propagated instead of returning an error dict (memory tools bypass `_execute_tool` exception handler). **Fix:** Explicit validation returning `{"status": "error", ...}`.

`update_memory` **tool whitespace** `content`**:** `if content:` evaluates `True` for whitespace-only strings, then

`update()` raises `ValueError`. **Fix:** Check `content.strip()` before adding to kwargs.

10. Validation & Error Handling

`update_memory` **category validation:** Could store categories like `"todo"` or `"task"` that never existed in the schema. **Fix:** Validates against the 6-category set.

`update_memory` `reminded_at` **validation:** Accepted any string including natural language, breaking SQL comparisons. **Fix:** Must be valid ISO 8601 or the special keyword `"now"`.

`_build_dynamic_memory_context()` **naive** `due_at:` `due_dt < now` raises `TypeError` when `due_dt` is naive. **Fix:** `due_dt` normalized to tz-aware via `.astimezone()` before comparison.

`get_all_knowledge(search=...)` **all-special-char:** Sanitizes to `None`, returning all items. **Fix:** Return `{"items": [], "total": 0}` immediately.

HTTP 422 vs 500 for dashboard inputs: `ValueError` raised in handler code becomes HTTP 500 instead of 422. **Fix:** Validation moved into Pydantic `@field_validator` methods on `KnowledgeCreate` and `KnowledgeUpdate`.

`update()` `reminded_at` **normalization:** Direct Python callers bypass the API Pydantic layer. **Fix:** `update()` normalizes naive `reminded_at` to tz-aware ISO 8601.

`_auto_store_error()` **empty error message:** Stored useless `"tool_name:"` entries. **Fix:** Returns early when `error_msg` is empty or whitespace-only.

11. Atomicity & Concurrency

`search()` **confidence bump lost-update:** Python-side arithmetic vulnerable to multi-process race. **Fix:** SQL-side `MIN(confidence + 0.02, 1.0)` for atomic increment.

`search()` **confidence bump partial commit:** Loop could commit partial batch if one UPDATE fails. **Fix:** Entire bump loop + `commit()` wrapped in `try/except: rollback`.

`store()` **FTS insert failure orphaned row:** Knowledge INSERT committed by next unrelated `commit()`. **Fix:** New-entry branch wrapped in `try/except: rollback; raise`.

`store()` **dedup branch FTS update failure:** Same orphaned-row pattern. **Fix:** UPDATE + FTS sync wrapped in `try/except: rollback; raise`.

`update()` **FTS sync failure:** Knowledge UPDATE committed without FTS sync. **Fix:** Lock block wrapped in `try/except: rollback; raise`.

`delete()` **FTS and knowledge DELETEs not atomic:** Ghost rows possible. **Fix:** Both DELETEs wrapped in `try/except: rollback; raise`.

`store_turn()` **and** `log_tool_call()` **no rollback:** INSERT could be committed inconsistently. **Fix:** Wrapped in `try/except: rollback; raise`.

`rebuild_fts()` / `prune()` **partial DELETE committed by next operation:** Multi-step SQL without rollback guard. **Fix:** Wrapped in `try/except: self._conn.rollback()`.

12. Dashboard & API

`rebuild_fts()` **and** `prune_memory()` **raw exceptions as HTTP 500:** No structured error response. **Fix:** `try/except Exception` raising `HTTPException(500, ...)` with structured detail.

`set_memory_context()` **stale system prompt:** Context switch didn't rebuild the cached system prompt. **Fix:** Calls `self.rebuild_system_prompt()` if available.

`apply_confidence_decay()` **never called on long-running servers:** Only triggered in

`reset_memory_session()`. **Fix:** `init_memory()` now calls it on startup.

`remember` / `update_memory` **silent content truncation:** LLM received no indication content was shortened. **Fix:** Append truncation note to response message.

13. Performance

`get_tool_summary()` **N+1 query:** Fetched most-recent failure per tool in a Python loop. **Fix:** Single SQL statement using `LEFT JOIN` with `ROW_NUMBER() OVER (PARTITION BY tool_name ...)`. Requires SQLite >= 3.25 (window functions).

Bounded list queries: `get_entities()` and `get_contexts()` accept `limit: int = 100`; `get_upcoming()` accepts `limit: int = 10`. All parameterized to prevent unbounded scans.

JSON corruption resilience: `_row_to_knowledge_dict()` and `_row_to_tool_dict()` now use `_safe_json_loads()` that returns `None` on parse failure instead of crashing the entire query.

Implementation Order

1. `memory_store.py` -- Add `embedding BLOB`, `consolidated_at`, new methods (hybrid search, consolidation, embedding storage)
2. `memory.py` -- LLM extraction, hybrid search hooks, consolidation, embedding backfill
3. `agent.py` -- Mixin prompt check + post-query hook (unchanged from original design)
4. **Unit tests** -- Full coverage including hybrid search, extraction, consolidation, rollback discipline
5. **Integration tests** -- MemoryMixin lifecycle (prompt injection, tool logging, conversation storage, LLM extraction)
6. **Wire into ChatAgent** -- First consumer: `class ChatAgent(MemoryMixin, Agent, ...)`
7. `discovery.py` -- System discovery module (file system, git, apps, browser, email)
8. **Bootstrap flow** -- Conversational onboarding + discovery review
9. `ui/routers/memory.py` -- Add consolidate endpoint, rebuild-embeddings endpoint, all dashboard endpoints
10. `MemoryDashboard.tsx` -- Frontend dashboard with consolidation status, embedding coverage stats

---

Future Extensions

The memory architecture is designed to support the full AI PC assistant vision. These features build on the schema and APIs defined above -- no redesign needed.

Extension	How memory supports it	Fields used
Scheduler / Wake-up	Scheduler polls `GET /api/memory/upcoming?days=0`. Triggers OS notification or new agent session for overdue items. Zero memory changes needed.	`due_at`, `reminded_at`
Email Triage	Email contacts as entities (`person:sarah_chen`). Email prefs as knowledge. Sensitive emails flagged. Tool history tracks email API calls.	`entity`, `sensitive`, `context="email"`
Browser Automation	Learned workflows stored as `category='skill'`. Browser errors auto-learned. Per-site preferences tracked.	`skill`, `error`, `entity="site:github.com"`
Multi-Agent Workspaces	Each agent uses a different `context`. System prompt filters by context. No cross-contamination.	`context`
App Integrations	Each app is an entity (`app:vscode`, `service:slack`). App-specific prefs and errors tracked.	`entity`, `preference`, `error`
Contact Management	People as entities with accumulated facts. `recall(entity="person:X")` returns full profile.	`entity` pattern
Knowledge Graph	`entity` tags are today's primitive. Full KG with typed relations (`person:X ->works_on-> project:Y`) is a future layer that adds a `relations` table.	`entity` -> future `relations` table
Learning Loop	Tool history and knowledge confidence data can feed model fine-tuning or	

GRPO reinforcement once a training pipeline exists. | `tool_history`, `confidence` | | **ColBERT / Late-Interaction Retrieval** | If personal-scale knowledge grows beyond single-vector retrieval quality, the embedding BLOB column and FAISS index can be swapped for a ColBERT index. | `embedding` column | | **Cloud Sync** | `memory.db` can be synced via file-level replication (e.g., Syncthing, OneDrive). True multi-device merge requires conflict resolution on the knowledge table. | All tables | | **Encrypted at rest** | AES-256-GCM encryption of `memory.db`. Required for multi-user accounts and enterprise. Zero schema changes · encryption is at the SQLite level (SQLCipher or custom VFS). | All tables | | **Multi-modal memory** | Images, screenshots, diagrams, voice memos stored as knowledge items. Multi-modal embeddings (CLIP/SigLIP) for cross-modal search ("find the whiteboard photo about the architecture"). | `embedding` (multi-modal), `metadata` (file refs) | | **Graph traversal** | Typed entity relationships (`person:X ·works_on· project:Y`) A-MEM Zettelkasten linking. MAGMA multi-graph. Enables "who works with Sarah?" queries. | Future: v3 |